

List of Functional/Logical and Security bugs for Flask App which is created in python.

Steps to Execute the Tests in Postman:

1. Install Postman if you don't already have it installed from Postman Official Website.
2. Launch Postman and follow these steps:
 - Create a New Request: Click on New -> Request to start a new test.
 - Set the URL: Enter the correct URL for the test (e.g., `http://127.0.0.1:5000/client_registration`).
 - Set the Request Type: Use the appropriate HTTP method (POST, GET, etc.).
 - Add Body Parameters (For POST requests): Go to the Body tab in Postman and select form-data. Add the necessary parameters (e.g., `fullName`, `userName`, etc.).
 - Send the Request: Click on the Send button to send the request to your Flask server.
 - Inspect the Response: Check the response body and status code to ensure everything works as expected.
3. Repeat for other scenarios: For each test scenario, modify the request body or parameters accordingly and verify the response.

Sample code for Above #####

1. Testing /client_registration (User Registration)

Scenario 1: Valid Registration

- Request Type: POST
- URL: `http://127.0.0.1:5000/client_registration`
- Request Body (Form Data):
 - `fullName`: John Doe
 - `userName`: johndoe
 - `email`: johndoe@gmail.com
 - `password`: test1234
 - `phone`: +1234567890
- Expected Response:
 - `{"msg": "User Registered"}` if the user is registered successfully.

Bug#	Bug Type	Description	Risk Score
1	Functional	Inconsistent return values in API responses	6
2	Functional	Incomplete data validation for <code>client_registration</code>	5
3	Functional	Missing error handling for <code>client_login</code>	6
4	Functional	User role handling and	7

		authorization issues	
5	Functional	Token expiration and validation issues	7
6	Functional	Unnecessary database queries in <code>/products</code>	5
7	Functional	Inconsistent HTTP method for <code>/client_login</code>	5
8	Functional	Missing database transaction rollback	5

List of Security Issues:

1. Password Storage in Plain Text (High Risk - 9/10)
2. Weak JWT Secret (High Risk - 8/10)
3. SQL Injection Vulnerability (High Risk - 9/10)
4. JWT Token Without Signature Verification (High Risk - 8/10)
5. Missing Input Validation (Medium Risk - 6/10)